agronomy

MDPI

Article

Improved Lightweight YOLOv8 Model for Rice Disease Detection in Multi-Scale Scenarios

Jinfeng Wang [1,*], Siyuan Ma [1], Zhentao Wang [2], Xinhua Ma [1], Chunhe Yang [1], Guoqing Chen [1] and Yijia Wang [3,*]

[1] College of Engineering, Northeast Agricultural University, Harbin 150030, China
[2] College of Mechanical and Electrical Engineering, Shihezi University, Shihezi 832003, China
[3] Department of Engineering Management, Northeast Agricultural University, Harbin 150030, China
* Correspondence: jinfeng_w@126.com (J.W.); yijiaw@neau.edu.cn (Y.W.); Tel.: +86-130-4515-6938 (J.W.); +86-156-3600-9336 (Y.W.)

Abstract: In response to the challenges of detecting rice pests and diseases at different scales and the difficulties associated with deploying and running models on embedded devices with limited computational resources, this study proposes a multi-scale rice pest and disease recognition model (RGC-YOLO). Based on the YOLOv8n network, which includes an SPPF layer, the model introduces a structural reparameterization module (RepGhost) to achieve implicit feature reuse through reparameterization. GhostConv layers replace some standard convolutions, reducing the model's computational cost and improving inference speed. A Hybrid Attention Module (CBAM) is incorporated into the backbone network to enhance the model's ability to extract important features. The RGC-YOLO model is evaluated for accuracy and inference time on a multi-scale rice pest and disease dataset, including bacterial blight, rice blast, brown spot, and rice planthopper. Experimental results show that RGC-YOLO achieves a precision (P) of 86.2%, a recall (R) of 90.8%, and a mean average precision at Intersection over Union 0.5(mAP50) of 93.2%. In terms of model size, the parameters are reduced by 33.2%, and GFLOPs decrease by 29.27% compared to the base YOLOv8n model. Finally, the RGC-YOLO model is deployed on an embedded Jetson Nano device, where the inference time per image is reduced by 21.3% compared to the base YOLOv8n model, reaching 170 milliseconds. This study develops a multi-scale rice pest and disease recognition model, which is successfully deployed on embedded field devices, achieving high-accuracy real-time monitoring and providing valuable reference for intelligent equipment in unmanned farms.

Keywords: rice pest and disease recognition; object detection; YOLOv8n; lightweight; reparameterization

check for updates

Academic Editor: Peng Fu

Received: 6 January 2025
Revised: 8 February 2025
Accepted: 9 February 2025
Published: 11 February 2025

Citation: Wang, J.; Ma, S.; Wang, Z.; Ma, X.; Yang, C.; Chen, G.; Wang, Y. Improved Lightweight YOLOv8 Model for Rice Disease Detection in Multi-Scale Scenarios. *Agronomy* **2025**, *15*, 445. https://doi.org/10.3390/agronomy15020445

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Rice, as a major staple crop in China, plays a vital role in ensuring national food security [1,2]. Its stable increase in production is of great significance. However, pests and diseases are key factors that affect rice yield, and timely detection and management are crucial to minimizing yield loss [3]. Currently, the early warning of rice pests and diseases primarily relies on manual identification by farmers and plant protection experts. This method is not only time-consuming and labor-intensive but also limited by the availability of human resources [4]. Therefore, there is an urgent need for an intelligent, automated rice pest and disease recognition method.

Machine learning [5] has provided more possibilities for pest and disease identification. Pantazi et al. [6] proposed a method using local binary patterns for feature extraction and

automatic identification of grape leaf diseases (healthy, downy mildew, powdery mildew, and black rot), achieving a classification success rate of 95%. Shin et al. [7] used three feature extraction techniques (HOG, SURF, and GLCM) combined with two supervised machine learning methods (ANN and SVM) for the recognition of strawberry powdery mildew, with the highest classification accuracy of 94.34% achieved using the combination of ANN and SURF. Abbaspour-Gilandeh et al. [8] used image processing techniques, sparse methods, artificial neural networks, and MATLAB software (R2013a, MathWorks, Natick, MA, USA) to classify apple pests and diseases with an accuracy of 90%. The above research shows that the application of machine learning in the field of agricultural disease identification has achieved good results. However, in the field of rice pest and disease identification, the diversity of diseases and the varying feature scales make feature extraction challenging, presenting considerable difficulties for traditional image processing methods. In recent years, with the development of deep learning, more possibilities have been provided for rice pest and disease detection [9]. Huang et al. [10] used the GoogLeNet model for panicle blast disease detection with an accuracy of 92%. Lin et al. [11] proposed a rice planthopper image classification method based on transfer learning and Mask R-CNN, with an average recognition accuracy of 92.3%. Bari et al. [12] used the Faster R-CNN model to recognize rice leaf diseases (rice blast, brown spot, and Hispa) with an accuracy of 99.25%. Zhang et al. [13] used ResNet as the backbone network to develop a rice disease detection algorithm, achieving the highest accuracy of 99.87% for two rice diseases (brown spot and leaf blast). Haruna et al. [14] achieved accurate recognition of rice diseases (bacterial blight, rice blast, brown spot, and Tungro) after data augmentation using the Single Shot MultiBox Detector (SSD) algorithm, with a mAP of 91%. These studies used algorithms like Fast R-CNN and SSD to achieve relatively accurate feature extraction, but the large model size and slow computational speed require high computational resources. Therefore, how to achieve model lightweight while ensuring accuracy is a major challenge in the field of rice pest and disease detection.

To enable the deployment of object detection models on devices with limited computational resources [15], extensive efforts have been made by researchers. Jiao et al. [16] proposed a litchi recognition model based on YOLOv3-SPP, with channel pruning and layer pruning for model compression, achieving a compression rate of 96.8% and an average accuracy of 95.3%. Lawal [17] used the ShuffleNetv2 network to optimize YOLOv5, achieving fruit detection with a detection mAP50 of 89.3% and a model weight of 3 MB. Firdiantika et al. [18] used GhostConv to optimize YOLOv7, achieving a model with 30 M parameters and a detection accuracy of 99.6%. It is worth noting that the introduction of lightweight modules during model improvement may lead to a decrease in model accuracy. Wu et al. [19] propose a lightweight module (LW) based on the concept of depthwise separable convolution. Ablation experiments demonstrate that introducing the LW module into the original model results in a 2.1% decrease in mAP. Huang et al. [20] used lightweight networks like MobileNetV3 and lightweight modules such as SPPFCSPC-GS to optimize YOLOv7. Ablation experiments showed that although the introduction of these modules led to a significant reduction in model parameters, the accuracy decreased by 0.6 percentage points compared to the original model. Therefore, how to apply deep learning and lightweight improvement strategies to achieve accurate rice pest and disease recognition and deployment on edge devices remains a major challenge.

To address these issues, the main research contributions are as follows: (1) A dataset was constructed containing four types of rice pests and diseases: rice bacterial blight, rice blast, brown spot, and planthopper. (2) A multi-scale lightweight rice pest and disease detection model, RGC-YOLO, was proposed. This model enhances the YOLOv8n network by incorporating C2f-RepGhost, GhostConv, and CBAM layers. (3) The effectiveness of the

model improvements was validated through ablation experiments, comparative tests, and deployment experiments. The ultimate goal is to deploy the model on devices with low computational resources that can accurately recognize rice diseases at multiple scales.

2. Materials and Methods

2.1. Data Collection and Data Augmentation

Four common rice diseases and pests (Bacterial Blight, Rice Blast, brown spot, and Planthopper) on rice leaves were selected as the research subjects. The self-built dataset is sourced from (1) RGB images of rice leaf diseases taken at the Rice Research Base in Fangzheng County, Harbin City, Heilongjiang Province, which includes three types of rice leaf diseases, with a total of 1000 images; (2) images of Rice Planthopper taken at the experimental base of Northeast Agricultural University in Xiangfang District, Harbin City, Heilongjiang Province, with a total of 243 images. In total, 1243 images were collected in natural field conditions. Due to the issues with mixed diseases, blurriness, and interference in the images captured in natural conditions, image samples were screened, and 697 images were retained. To prevent data leakage [21], we strictly controlled the order of dataset partitioning and data augmentation. For the self-collected dataset, 235 images were reserved as the test set, while the remaining 462 images underwent data augmentation. Additionally, several data augmentation techniques [22] were applied to the retained images, including brightness enhancement, rotation, mirroring, flipping, and translation. After augmentation, to ensure the ratio of training, validation, and test sets remained 8:1:1, we excluded images that were unrecognizable. As a result, 1490 augmented images and 462 original images were merged for further partitioning into training and validation sets. Some of the images in the dataset are shown in Figure 1.

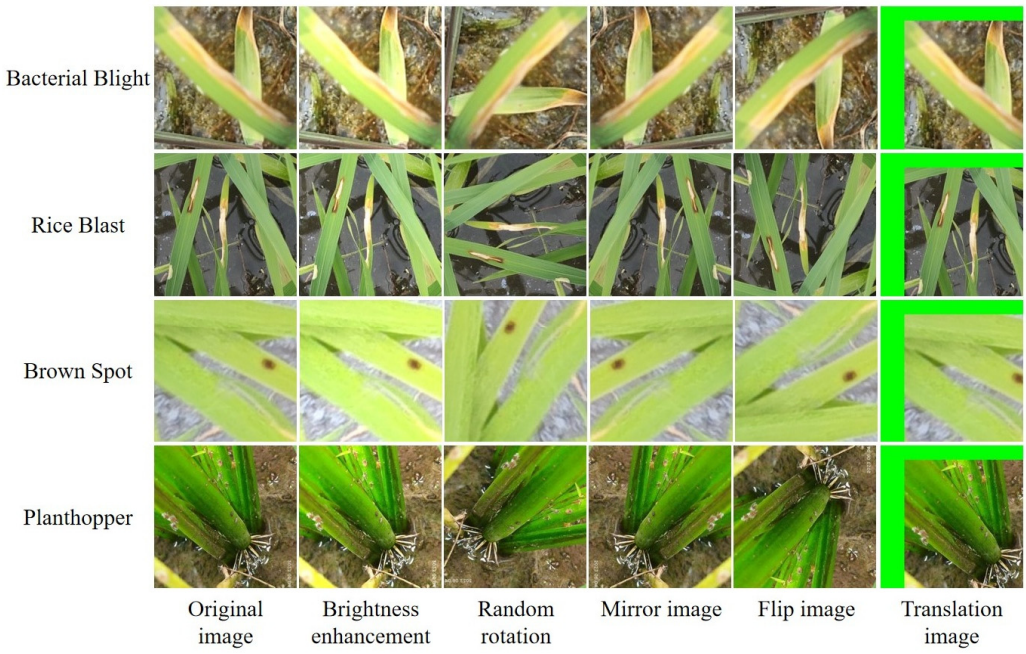


Figure 1. Original and data-augmented images of rice diseases and pests from the self-built dataset.

2.2. Construction and Division of Multi-Scale Dataset

To further improve the model's generalization ability and recognition accuracy, the augmented dataset was combined with the publicly available "Rice Disease Datasets" from the Kaggle website (https://www.kaggle.com, accessed on 9 September 2023). The public dataset used in this study did not undergo data augmentation. Therefore, the public dataset was split into training, validation, and test sets in an 8:1:1 ratio. Table 1 shows the

detailed composition of the dataset, including the public dataset, self-captured images, and augmented images.

Table 1. Composition and Sources of the Dataset.

Data Source	Kaggle	Original Image	Augmented Image	Total
Number	4552	697	1490	6739

The images were manually annotated using the labelimg tool. During the annotation process, four different rice diseases and pests were labeled, and the annotation information was saved in XML format and then converted into TXT file format. This successfully constructed the rice pest and disease dataset. The dataset was divided into training, validation, and test sets in an 8:1:1 ratio. Table 2 provides a detailed breakdown of the four labels in each dataset.

Table 2. Composition of Label Counts in the Rice Diseases and Pests Dataset.

Classes	Training Set	Validation Set	Test Set	Total
Bacterial Blight	1397	203	207	1807
Rice Blast	1734	217	182	2133
Brown Spot	1741	239	211	2191
Planthopper	2241	305	290	2836
Total	7113	964	890	8967

The proportion of the Ground Truth Bounding Box that occupies the image reflects the scale information of the lesions. Therefore, the proportion of the Ground Truth Bounding Box size is used as the basis for multi-scale division. The Ground Truth Bounding Box size information is shown in Figure 2. The content shown in Figure 2b is the specific content of the XML file generated by the LabelImg program, which is written in Python, for annotating the images in the dataset. The calculation method for the proportion of the Ground Truth Bounding Box size is as follows:

$$S_{label} = \frac{width_{box}}{width_{img}} \times \frac{height_{box}}{height_{img}} \quad (1)$$

where $width_{img}$ represents the width of the image, $height_{img}$ represents the height of the image, $width_{box}$ represents the width of the Ground Truth Bounding Box, and $height_{box}$ represents the height of the Ground Truth Bounding Box.

The proportion of ground truth box sizes was chosen to demonstrate the scale differences of the four types of pest and disease target regions in the dataset. To visually present this difference, a boxplot of the Ground Truth Bounding Box size proportion is shown in Figure 3. In the boxplot, the upper and lower edges represent the maximum and minimum values of the data, while the upper and lower edges of the box correspond to the 25th and 75th percentiles of the data points, respectively. The horizontal line inside the box represents the median and data points outside the box are considered outliers.

By observing Figure 3, it can be seen that the Ground Truth Bounding Box size span of Bacterial Blight is relatively large, and the data distribution is quite dispersed. The Ground Truth Bounding Box sizes of Rice Blast and brown spot disease have moderate dispersion, while the Ground Truth Bounding Box sizes of Rice Planthopper are relatively concentrated. The average Ground Truth Bounding Box size proportion for Bacterial Blight is 0.55, for Rice Blast, is 0.26, for brown spot disease is 0.29, and for Rice Planthopper is only 0.003. Therefore, in this dataset, Bacterial Blight can be considered a large-scale label,

Rice Blast, and brown spot disease are medium-scale labels, and Rice Planthopper is a small-scale label.

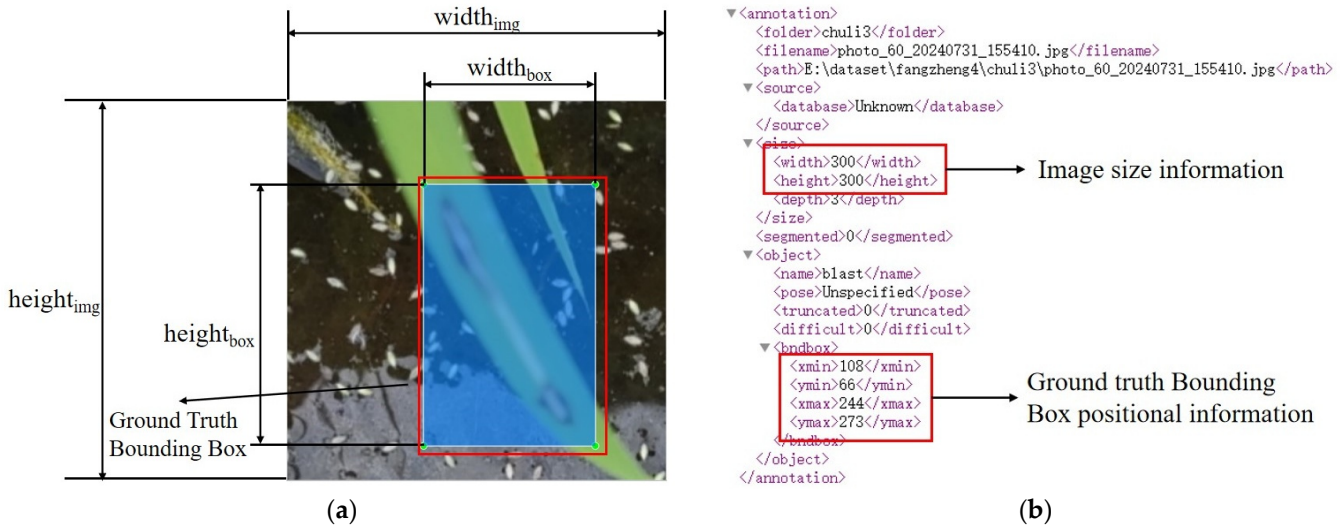


Figure 2. Dataset Ground Truth Bounding Box information schematic. (**a**) Dataset Ground Truth Bounding Box dimension information; (**b**) Dataset Label Information.

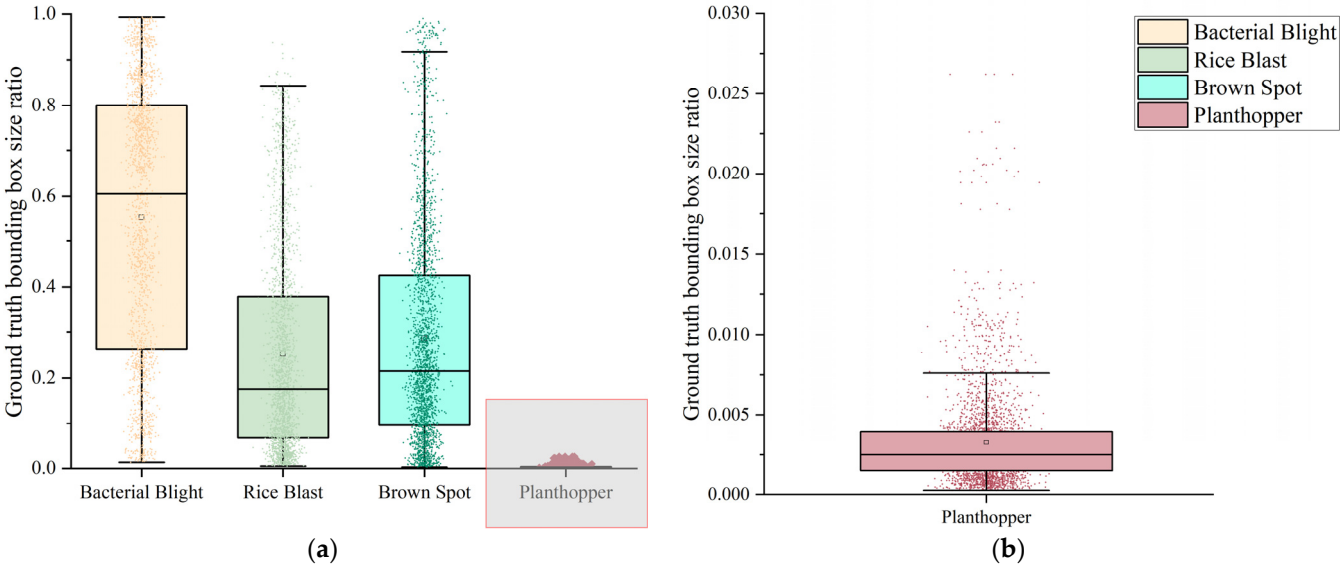


Figure 3. Different label Ground Truth Bounding Box size proportions. (**a**) The proportion diagram of the Ground Truth Bounding Box size of the four diseases and insect pests; (**b**) The proportion diagram of the Ground Truth Bounding Box size of rice planthoppers.

3. Rice Disease and Pest Identification Algorithm and Improvement

3.1. Overview of Model Architecture

The YOLOv8n model is the smallest model in terms of parameters and the fastest in terms of running speed within the YOLOv8 series, with the potential for deployment on edge devices [23]. Therefore, YOLOv8n was selected as the base model, and improvements were made on this foundation, leading to the proposed RGC-YOLO model, as shown in Figure 4.

The specific improvements include (1) Replacing some traditional Conv layers with GhostConv layers to reduce the generation of redundant feature maps, thereby reducing the model's parameter size; (2) Optimizing the C2f layers in the backbone and neck networks by replacing the original C2f layers with C2f_RepGhost, further reducing the model

parameters and computational cost; and (3) Adding a CBAM layer before the SPPF layer in the backbone network to enhance the model's feature extraction ability. The detailed parameters of the improved network structure are shown in Table 3.

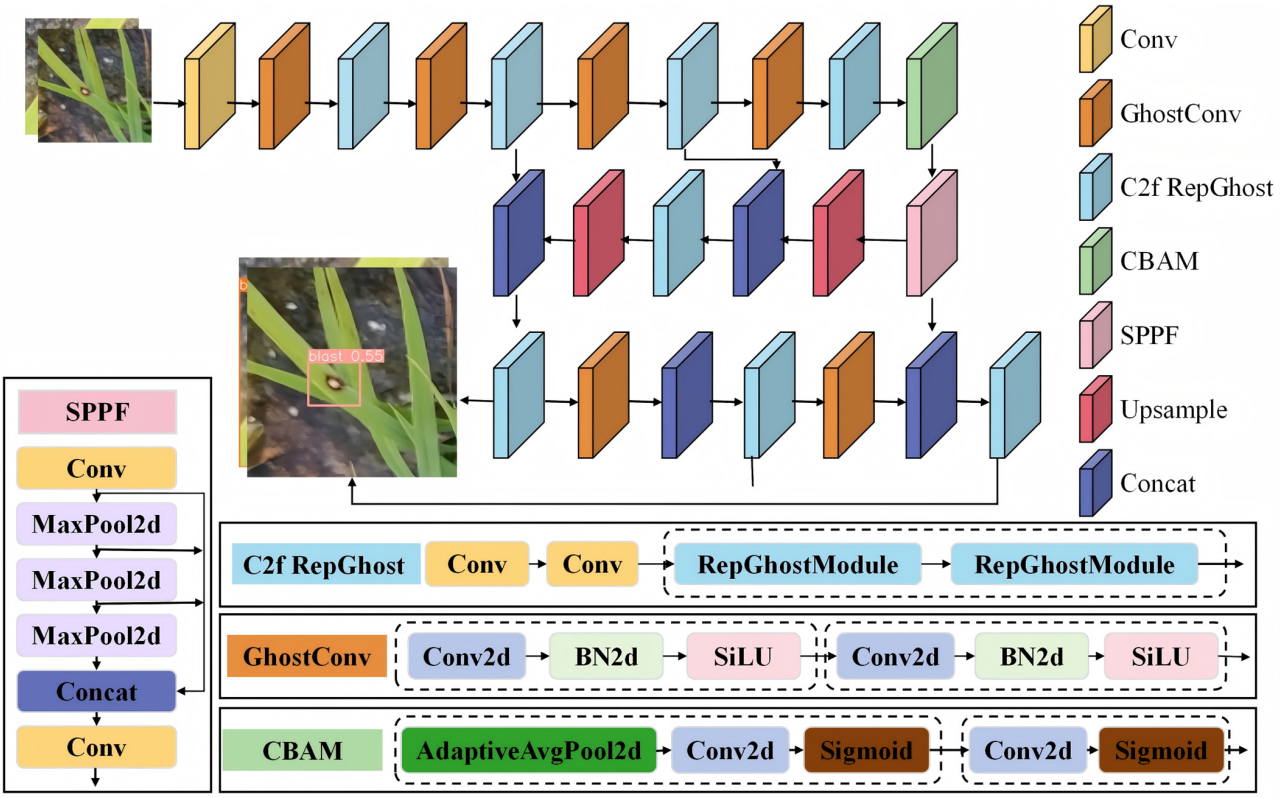


Figure 4. The improved YOLOv8n network structure. Note: Conv represents ordinary convolution, GhostConv is ghost convolution, C2f RepGhost is an improved reparameterized module, CBAM is a hybrid attention mechanism module, SPPF is a spatial pyramid pool structure, Upsample is upsampling, and concat is tensor connection. MaxPool2d is a maximum pooling operation; RepGhostModule is a heavily parameterized module.

Table 3. Improve network structure parameters.

Layer	Module	Number of Channels, Kernel Size, and Stride	Parameters
0	Conv	3, 16, 3, 2	464
1	GhostConv	16, 32, 3, 2	2768
2	C2f_RepGhost	32, 32, True	3680
3	GhostConv	32, 64, 3, 2	10,144
4	C2f_RepGhost	64, 64	27,008
5	GhostConv	64, 128, 3, 2	38,720
6	C2f_RepGhost	128, 128	103,168
7	GhostConv	128, 256, 3, 2	151,168
8	C2f_RepGhost	256, 256, True	201,472
9	CBAM	256	65,890
10	SPPF	256, 256, 5	164,608
11	Upsample	None, 2, 'nearest'	0
12	Concat	1	0
13	C2f_RepGhost	384, 128	84,352
14	Upsample	None, 2, 'nearest'	0
15	Concat	1	0
16	C2f_RepGhost	192, 64	21,696
17	GhostConv	64, 64, 3, 2	19,360
18	Concat	1	0
19	C2f_RepGhost	192, 128	59,776
20	GhostConv	128, 128, 3, 2	75,584
21	Concat	1	0
22	C2f_RepGhost	384, 256	234,240
23	Detect	3, 64, 128, 256	751,897
total			2,015,995

3.2. Model Improvement

3.2.1. GhostConv

GhostConv [24] utilizes existing feature maps to generate more "ghost" feature maps through low-cost linear transformations, thereby improving the computational efficiency of the network. This approach aims to reduce the computational burden and increase resource utilization, making it suitable for scenarios with limited computational resources, such as embedded devices. In deep neural networks, rich feature maps are crucial for extracting comprehensive features for different labels. The Ghost module minimizes redundant feature maps while preserving the total number of generated feature maps. By producing additional feature maps with fewer parameters, this module effectively lowers computational overhead without compromising feature extraction efficiency. The detailed network architecture is illustrated in Figure 5. The computational cost of conventional convolution, denoted as J_C, is given by Equation (2):

$$J_c = n \times h' \times w' \times c \times k \times k \tag{2}$$

where n represents the number of feature maps; h′ represents the height of the output feature map after one convolution; w′ represents the width of the output feature map after one convolution; c represents the number of input channels; and k represents the size of the conventional convolution kernel.

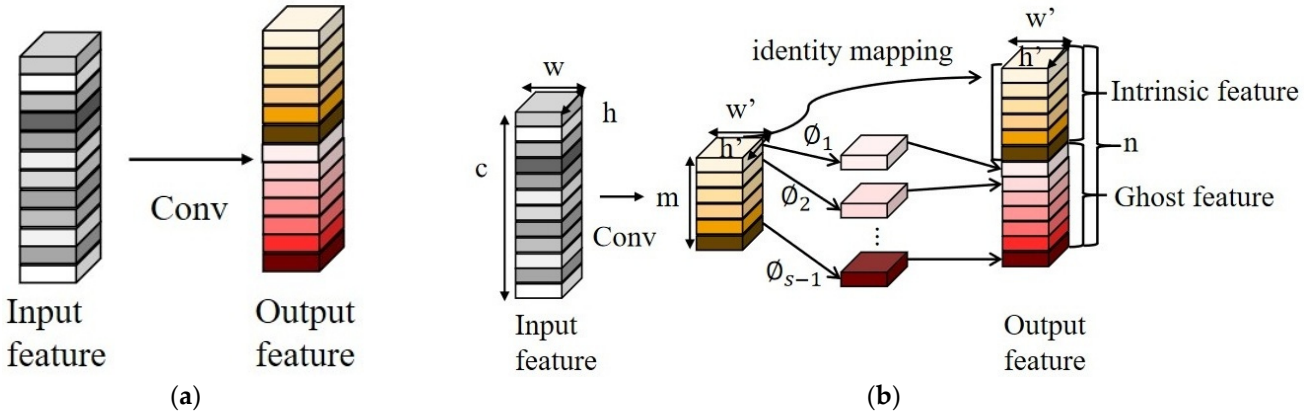


Figure 5. Generating sketch map of feature map: (**a**) Conv feature map generation schematic diagram; (**b**) Ghostconv feature map generation schematic diagram.

The computational cost of Ghost convolution, denoted as J_G, is given by Equation (3):

$$J_G = \frac{n}{s} \times h' \times w' \times c \times k \times k + (s-1) \times \frac{n}{s} \times h' \times w' \times d \times d \tag{3}$$

where n/s represents the number of output channels after the first transformation; s represents the number of features generated by the Ghost convolution; and d represents the size of the linear convolution kernel.

The computational cost ratio between conventional convolution and Ghost convolution, denoted as R_J, is given by Equation (4):

$$R_J = \frac{n \times h' \times w' \times c \times k \times k}{\frac{n}{s} \times h' \times w' \times c \times k \times k + (s-1) \times \frac{n}{s} \times h' \times w' \times d \times d} \tag{4}$$

When d = k and s << c,

$$R_J = \frac{c \times k \times k}{\frac{1}{s} \times c \times k \times k + \frac{s-1}{s} \times d \times d} \approx \frac{s \times c}{s + c - 1} \approx s \tag{5}$$

In summary, the computational cost of conventional convolution is approximately s times that of Ghost convolution. This reduction in computational cost decreases resource consumption and improves the running speed.

3.2.2. RepGhost Lightweight Module (C2f RepGhost)

To balance model accuracy and model size, the RepGhost lightweight module [25] was used to further optimize the model. The RepGhost module reparameterizes the traditional Ghost network, effectively reducing computational complexity while preserving the efficient feature extraction capability of the Ghost module and optimizing parameter usage. As a result, it greatly reduces the model size and computational cost while preserving high accuracy. The specific network diagram is shown in Figure 6.

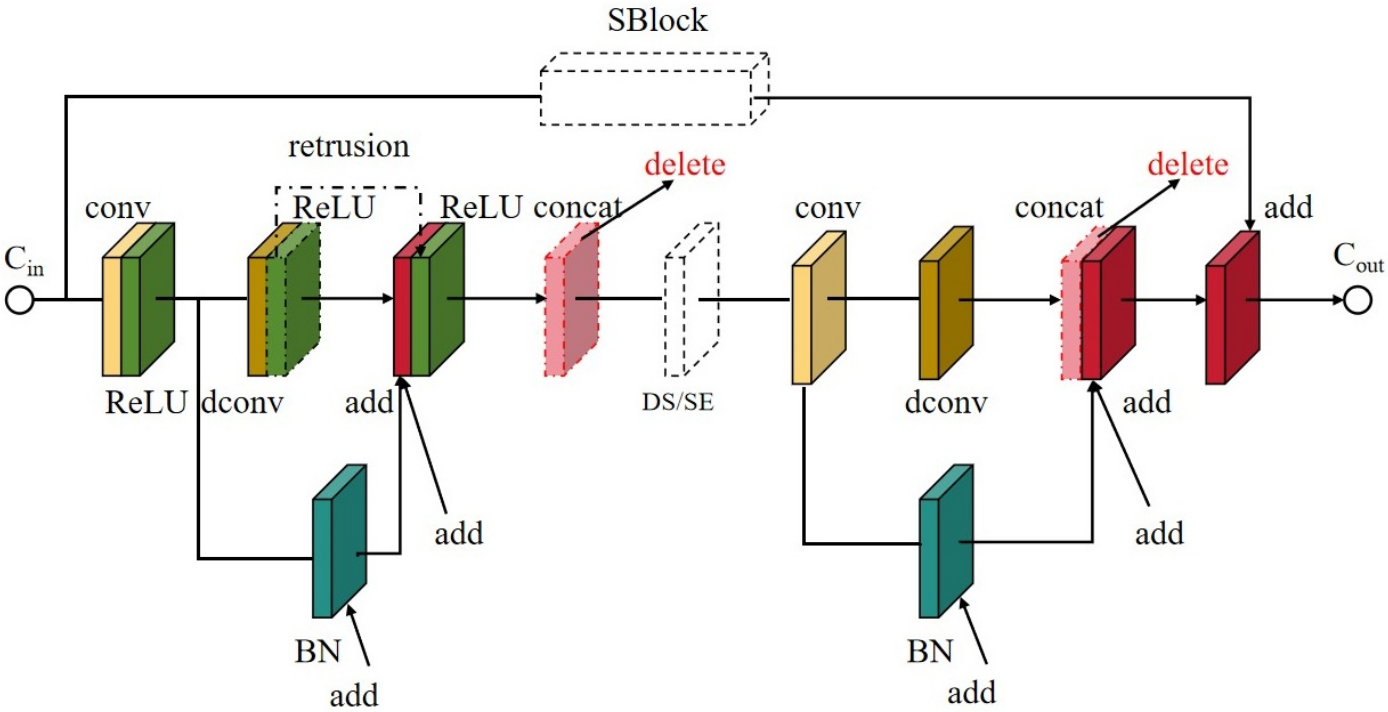


Figure 6. Internal structure of the RepGhost module and its improvements over the Ghost module. Note: cv (conv) is an ordinary convolution, ReLU is an activation function, concat is a tensor join, dconv is a deeply separated convolution, add is an add operation, SBlock: shortcut block, DS: Undersampling layer, SE: Squeenze and Excitation modules. RG-bneck: RepGhost bottleneck. Dashed blocks are inserted only when necessary. Cinand Cout represents the input and output channels of the bottleneck, respectively.

To achieve feature reuse, the RepGhost lightweight module mainly makes the following adjustments:

(1) Replacing the tensor concatenation operator with an addition operator: In the original network, feature reuse was achieved through tensor concatenation, which retained the existing feature maps and left the information processing to other operations. However, tensor concatenation operations incur additional computation when reused on hardware devices. Unlike tensor concatenation, the addition operator allows feature fusion, and the fusion process is completed in the weight space without introducing any external inference time, making the final architecture more efficient than concatenation. Based on this concept, we propose the RepGhost module, which achieves feature reuse through structural reparameterization.

(2) Shifting the position of the ReLU activation function: The ReLU function introduces non-linearity. In the original order, each transformed feature map was passed through

non-linear activation separately, while in the modified order, the entire fused feature map was passed through non-linear activation. This reduces the computational cost because the ReLU function only needs to be applied once to the fused feature map, rather than being applied separately to each transformed feature map. The ReLU function outputs x when $x > 0$ and outputs 0 otherwise. The expression is as follows:

$$f(x) = \max(0, x) \tag{6}$$

where x represents the input feature.

(3) Reparameterization by adding Batch Normalization (BN) in the branch: The BN layer normalizes the input data, reducing the model's dependency on the data distribution and thereby improving the model's generalization ability. The addition of the BN layer increases the network complexity during model training, but during model inference, the parameters of the BN layer are merged into the convolutional layer to improve the forward inference speed and reduce memory or GPU memory usage. The principle of the BN layer is as follows: in each training iteration, the input is first normalized by subtracting its mean and dividing by its standard deviation, both of which are based on the current mini-batch. Then, the scaling factor and the shifting factor are applied. The batch normalization process is derived from the following formula:

$$\mathrm{E}\left(X^{\mathrm{t}-1}\right) = \frac{1}{\mathrm{n}} \sum_{\mathrm{i}=1}^{\mathrm{n}} X_{\mathrm{i}}^{\mathrm{t}-1} \tag{7}$$

$$\mathrm{D}\left(X^{\mathrm{t}-1}\right) = \frac{1}{\mathrm{n}-1} \sum_{\mathrm{i}=1}^{\mathrm{n}} \left[X_{\mathrm{i}}^{\mathrm{t}-1} - \mathrm{E}\left(X^{\mathrm{t}-1}\right)\right]^2 \tag{8}$$

$$\hat{X}_i^t = \frac{X_{\mathrm{i}}^{\mathrm{t}-1} - \mathrm{E}\left(X^{\mathrm{t}-1}\right)}{\sqrt{\mathrm{D}(X^{\mathrm{t}-1}) + \varepsilon}} \tag{9}$$

$$X_{\mathrm{i}}^{\mathrm{t}} = \gamma^{\mathrm{t}} \hat{X}_{\mathrm{i}}^{\mathrm{t}} + \beta^{\mathrm{t}} \tag{10}$$

where E represents the mean; D represents the variance; $X_i^{\hat{t}-1}$ represents the normalized result; n is the batch size of the input; X_i^{t-1} is the i-th element in the output batch of the previous layer; ϵ is a small number used to prevent numerical instability caused by division by zero; γ is the scaling factor applied to the normalized data; β is the shifting factor applied to the normalized data; and X_i^t is the i-th element in the output batch of this layer.

3.2.3. Hybrid Attention Mechanism (CBAM)

CBAM [26,27] is an auxiliary component aimed at improving the efficiency of convolutional neural networks. It comprises two primary submodules: the Channel Attention Module (CAM) and the Spatial Attention Module (SAM), depicted in Figure 7. These two submodules enhance the model's ability to represent features by assigning attention weights to both the channel and spatial locations. Specifically, CAM focuses on the feature information of the dataset images, while SAM focuses on the spatial location information of the image. CAM compresses the spatial dimensions while preserving the channel dimensions, whereas SAM compresses the channel dimensions while maintaining the spatial dimensions. The input features undergo parallel max-pooling and average-pooling operations, followed by a compression-expansion transformation through a shared multilayer perceptron in the channel attenuation module. The output of the channel attention

is obtained through the sigmoid activation function, as shown in Equation (11), and is multiplied by the input features to restore the original size.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{11}$$

where σ represents the sigmoid function; x is the input feature.

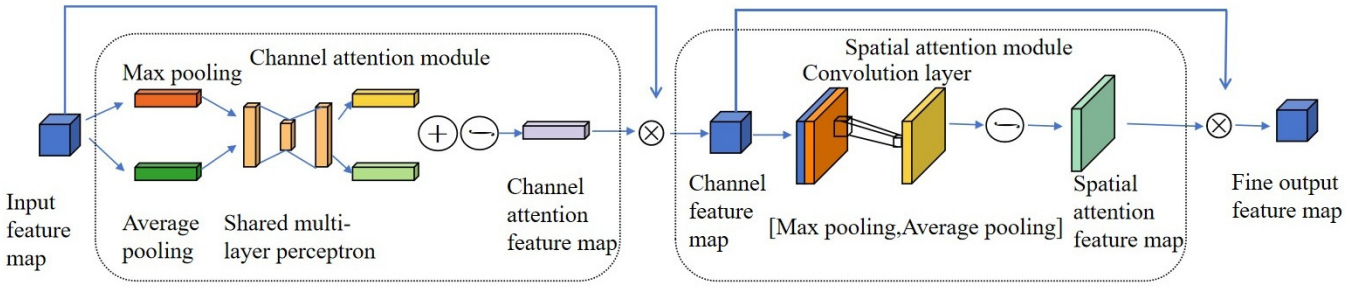


Figure 7. CBAM attention module structure.

In the Spatial Attention Module, the features obtained by multiplying the channel attention with the original image undergo both max-pooling and average-pooling operations simultaneously. These outputs are concatenated and then undergo a 7 × 7 convolution to generate a single-channel feature map. The sigmoid activation function is then applied to derive the output of the spatial attention, which is multiplied by the original image to restore its original size. The CAM (x) and SAM (x) can be summarized by the following equations:

$$\mathrm{M}_c = \sigma(\mathrm{MLP}(\mathrm{AvgPool}(x)) + \mathrm{MLP}(\mathrm{MaxPool}(x))) \tag{12}$$

$$\mathrm{M}_s = \sigma(\mathrm{Conv}[\mathrm{AvgPool}(x), \mathrm{MaxPool}(x)]) \tag{13}$$

where σ represents the sigmoid function; x represents the input feature.

4. Experiment and Analysis

4.1. Training Platform and Parameter Settings

The training and testing of the model in this study were performed on a computer equipped with an Intel® Core™ i7-12650H processor (2.30 GHz), an NVIDIA GeForce RTX 3050 GPU, and a Windows 11 (64-bit) operating system. The input image size was set to 640 × 640 × 3, with 300 iterations and a batch size of 16. The initial learning rate was set to 0.01, and the model was optimized using the SGD algorithm, which calculates an adaptive learning rate for each weight parameter. The deep learning framework used was PyTorch 2.1.1, and the programming language was Python 3.9.18. The code development and data analysis process was carried out using PyCharm 2021.3 (JetBrains, Amsterdam, The Netherlands, https://www.jetbrains.com/pycharm/, accessed on 9 January 2024) Community Edition. Data analysis and graphical visualization were performed using Origin 2024 (OriginLab, Northampton, MA, USA) software.

4.2. Evaluation Metrics

To comprehensively evaluate the detection performance of the improved algorithm RGC-YOLO, key metrics such as Precision, Recall, and mAP50 were used to measure the model's accuracy. To assess the lightweight nature of the model, metrics such as the number of parameters, Floating Point Operations per Second (FLOPs), and model memory usage were selected as evaluation criteria for the model size. In the deployment experiment,

inference time was chosen as an important indicator for evaluating model performance [28]. Inference time refers to the time taken from the preprocessed image input into the model until the model outputs the detection results. A shorter inference time indicates better real-time detection performance.

4.3. Ablation Experiment

To validate the effectiveness of GhostConv, C2f-RepGhost, and CBAM in improving the YOLOv8n model, ablation experiments were conducted on a multi-scale rice pest and disease dataset. The experiments involved eight models, comparing the performance changes between the improved models and the original model across various metrics. The experimental results are shown in Table 4. The results were analyzed to assess the impact of different modules on model improvement.

Table 4. Results of ablation tests.

Model	GhostConv	C2f RepGhost	CBAM	mAP50 (%)	P (%)	R (%)	Parameters	GFLOPS ($\times 10^9$)
Case1	-	-	-	90.8	86.2	88.7	3,011,612	8.2
Case2	√	-	-	90.4	86.2	87.9	2,726,828	7.5
Case3	-	√	-	90.1	85.7	87.1	2,225,564	6.3
Case4	-	-	√	90.9	86.2	88.3	3,077,518	8.3
Case5	√	√	-	90.1	85.4	86.1	**1,945,769** *	**5.8** *
Case6	-	√	√	92.4	86.1	89.5	2,291,454	6.4
Case7	√	-	√	92.7	85.7	**91.2** *	2,792,718	7.6
RGC-YOLO	√	√	√	**93.2** *	**88** *	90.8	2,011,659	**5.8** *

* Note: Bold values represent the best comparison result for the corresponding metric.

Case 2: Replacing the convolutional layers in the base YOLOv8n network with GhostConv, while keeping the first layer as regular convolution. The model showed a 9.46% reduction in parameter count and an 8.54% reduction in computational load. However, there was a drop in accuracy, with recall decreasing by 0.8 percentage points.

Case 3: Replacing the C2f layers in the YOLOv8n network with C2f-RepGhost layers. This resulted in a 26.10% reduction in parameters and a 23.17% reduction in computational load, but the accuracy metrics all declined, with recall dropping by 1.6 percentage points.

Case 4: Adding the CBAM module before the SPPF layer in the YOLOv8n network. This caused a 2.19% increase in parameter count and a 1.22% increase in computational cost. Notably, while mAP50 increased by 0.1, recall decreased by 0.4 percentage points.

In Cases 2, 3, and 4, each modification only introduced one module compared to the original model. Cases 2 and 3, which use lightweight modules, achieved reductions in parameters and computational load. However, the simplification of feature extraction operations led to a decrease in recall. The introduction of the CBAM module in Case 4 also resulted in reduced recall. The main function of CBAM is to emphasize important features and suppress irrelevant ones, but some important features are mistakenly regarded as irrelevant, leading to a decrease in model accuracy.

Case 5: Replacing the C2f layers with C2f-RepGhost layers and changing all regular convolution layers (except for the first layer) to GhostConv. This resulted in a 35.39% reduction in parameters and a 29.27% reduction in computational load, but all accuracy metrics declined, with recall decreasing by 2.6 percentage points.

Case 6: Replacing C2f layers with C2f-RepGhost layers and adding CBAM before the SPPF layer. This resulted in a 23.91% reduction in parameters and a 21.95% reduction in computational load. mAP50 increased by 1.6, while accuracy dropped by 0.1 percentage points, and recall improved by 0.8 percentage points.

Case 7: Replacing the convolutional layers in the base YOLOv8n network with GhostConv, while keeping the first layer as regular convolution, and adding CBAM before the SPPF layer. The model showed a 7.27% reduction in parameters and a 7.32% reduction in

computational load. mAP50 increased by 1.9, accuracy decreased by 0.5 percentage points, and recall increased by 2.5 percentage points.

In Cases 5, 6, and 7, two modules were combined to modify the original model. In Case 5, the combination of two lightweight modules led to notable reductions in both parameters and computational load. However, this combination resulted in the greatest decline in accuracy, with recall decreasing by 2.6 percentage points, indicating a severe issue with missed detections.

In Cases 6 and 7, which combine one lightweight module with CBAM, accuracy showed a slight decrease, but other metrics improved, and reductions in parameters and computational load were achieved. This indicates that combining a lightweight module with CBAM resulted in better model accuracy compared to adding CBAM alone. The lightweight module simplified the feature extraction process, reducing the number of features, while CBAM emphasized the extraction of important features, improving recall. However, some irrelevant features were still mistakenly identified as important, leading to a slight decrease in accuracy.

Comparing Case 6 and Case 7 reveals the differences between the Ghost and RepGhost networks. The Ghost network achieved greater improvements in recall and mAP but also resulted in a larger drop in accuracy. The RepGhost network, an improvement on Ghost, reduces the model's dependence on data distribution by introducing the BN layer, thus enhancing generalization. Therefore, the accuracy decline was smaller in the RepGhost network.

RGC-YOLO: This model incorporates all the improved modules. It reduces parameters by 33.20% and computational load by 29.27%, while improving overall accuracy. mAP increased by 2.4, accuracy improved by 1.8 percentage points, and recall increased by 2.1 percentage points. The inclusion of both lightweight modules minimized the extraction of irrelevant features, while the CBAM module emphasized important features, resulting in improved model accuracy. The results demonstrate that RGC-YOLO not only meets the lightweight design requirements but also shows improvements in accuracy.

Furthermore, the prediction results of the ablation experiment are shown in Figure 8. It can be observed that for larger-scale targets like Bacterial Blight, the models improved with the lightweight modules GhostConv and C2f_RepGhost exhibited some missed detections. However, after adding the hybrid attention module, the missed detection issue was notably reduced. On the other hand, due to the Intersection over Union (IoU) threshold of 0.7 for Non-Maximum Suppression (NMS), the Case 4 and Case 6 models experienced overlapping predicted boxes in the detection of brown spot disease. By introducing GhostConv, redundant feature maps were successfully reduced, effectively alleviating the issue of overlapping predicted boxes.

For the detection of small-scale targets like Rice Planthopper, Case 1 showed both missed detections and false detections. This was mainly due to the functionality of the C2f module, which refines and enhances features through multiple layers to capture more complex details. However, this also led to some unimportant features being misclassified as target features, causing false detections. To address this issue, the attention mechanism was added. This improved the model's ability to extract important target features and effectively suppress irrelevant features, resulting in more accurate detection of Rice Planthopper.

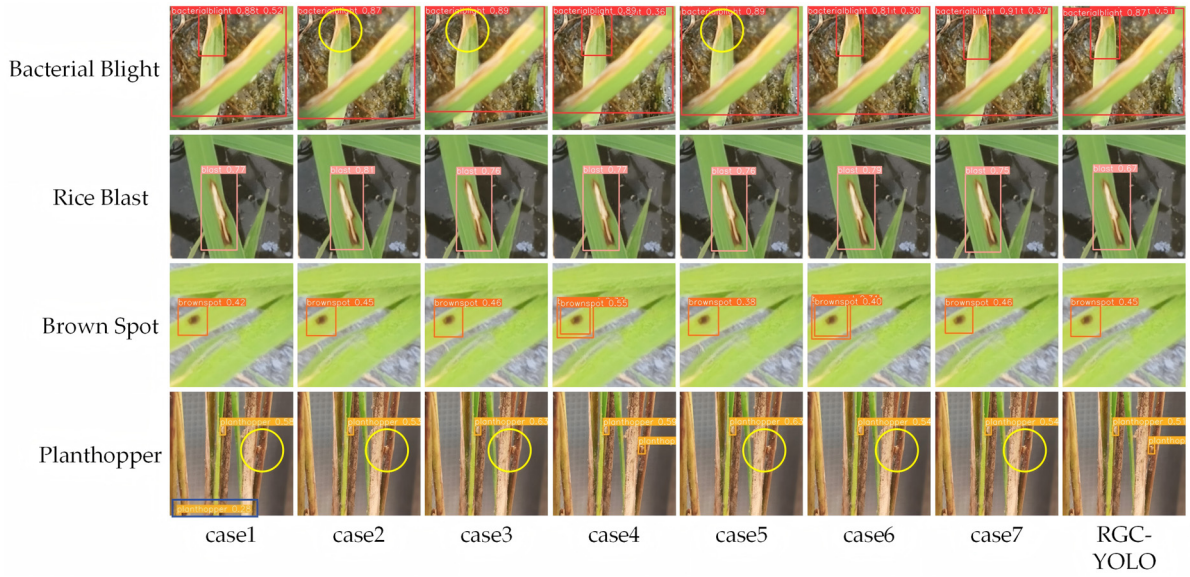


Figure 8. Illustration of prediction results on the test set by different models. Note: The red rectangle box, pink rectangle box, orange rectangle box, and yellow rectangle box in the figure are model prediction boxes, the yellow circle box is missed mark, and the blue rectangle box is false check mark.

4.4. Heatmap Analysis of the Attention Mechanism

To further analyze the performance of the lightweight improved model RGC-YOLO and the impact of different modules in the model improvements, heatmap testing was conducted on the test set. During the testing, a confidence threshold of 0.0001 was set to generate clear heatmaps. Heatmaps visually demonstrate which areas of the image have the greatest impact on the model's prediction results. To ensure the heatmap output is from the same output layer, for the network without the CBAM module, the heatmap from the 21st layer of the network was extracted for analysis. For the network with the CBAM module, the heatmap from the 22nd layer of the network was extracted for analysis. In the heatmap, the red areas show where the model focuses the most, indicating a strong contribution to detection. The yellow areas represent regions with less attention, while the blue areas reflect minimal impact on target detection, marking them as redundant information. The feature visualization results are shown in Figure 9. For Case 1, Case 2, Case 3, and Case 5, before adding the CBAM module, the model's feature extraction results were relatively scattered, and the areas of focus were not prominent. For Case 4, Case 6, Case 7, and RGC-YOLO, which included the CBAM module, the heatmaps show that the areas of focus are close to rectangular shapes. The Case 4 model is based on the YOLOv8n framework with the direct addition of the CBAM module. Visualization results indicate that the model's attention is predominantly concentrated in a rectangular region near the center of the image, with progressively lower attention levels toward the edges of the image. Case 6, based on Case 3 with the CBAM module added, enhanced attention to the edges. Case 7, which builds on Case 2 with the addition of the CBAM module, also increased attention to the edge areas. However, the feature extraction for Blast did not display clear distribution patterns. The use of GhostConv to replace Conv simplified the feature map generation process, resulting in incomplete feature extraction. However, RGC-YOLO, which replaced the C2f layer with the C2f RepGhost layer and replaced all Conv layers (except the first layer) with GhostConv, while also adding the CBAM module before the SPPF layer in the backbone network, reduced the generation of redundant feature maps and increased the attention to key features. The areas of focus were closely aligned with the pest and disease regions, resulting in notable improvements in feature extraction. Table 5 presents the training results of Case 1 and RGC-YOLO on the multi-scale dataset. The results show

that, compared to the base network YOLOv8n (Case 1), RGC-YOLO has improved the recognition accuracy for all four types of rice pests and diseases. Specifically, for large-scale diseases like Rice Bacterial Blight, the recognition accuracy did not improve. This is because the disease features of bacterial leaf blight are prominent, and the lesion area is relatively large, allowing the base network to sufficiently extract its lesion characteristics. However, for medium-scale diseases such as rice blasts and brown spots, the accuracy of RGC-YOLO improved. The challenge in recognizing Rice Blast lies in the variability of lesion shape during different stages of the disease, while brown spot disease has smaller Ground Truth Bounding. Box sizes but more distinct lesion features. RGC-YOLO effectively focuses on important features through the hybrid attention mechanism, suppressing irrelevant features, which led to a substantial improvement in recognition accuracy.

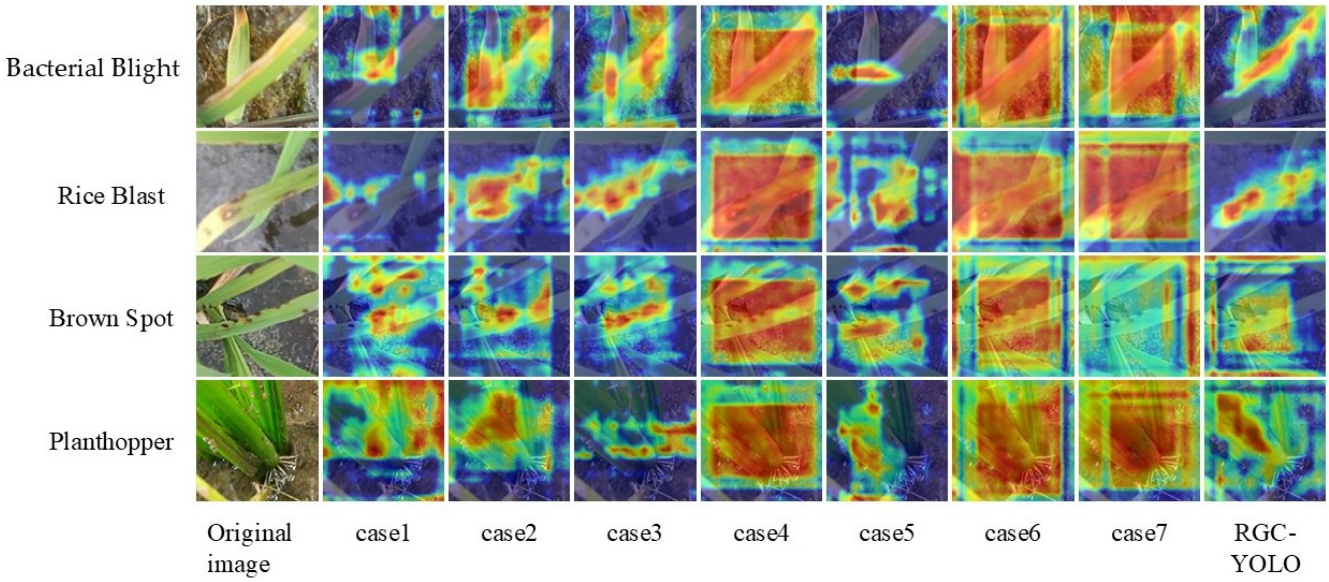


Figure 9. Heatmap of Image Feature Extraction Results by Different Models. Note: In the heatmap, the red areas show where the model focuses the most, indicating a strong contribution to detection. The yellow areas represent regions with less attention, while the blue areas reflect minimal impact on target detection, marking them as redundant information.

Table 5. Comparison of training accuracy for different rice diseases and pests between the improved and original models.

Classes	Model	mAP50 (%)	Recall (%)	Precision (%)
Bacterial Blight	Case1	97.6	95	**91.8** *
	RGC-YOLO	**98.2** *	**96.7** *	91.3
Rice Blast	Case1	87.6	83.6	80.9
	RGC-YOLO	**89.9** *	**85.9** *	**84** *
Brown Spot	Case1	87.6	79.9	84.6
	RGC-YOLO	**93.2** *	**89.5** *	**88** *
Planthopper	Case1	87.5	85.9	84.2
	RGC-YOLO	**91.6** *	**91.1** *	**89** *
All	Case1	90.1	86.1	85.4
	RGC-YOLO	**93.2** *	**90.8** *	**88** *

* Note: Bold values represent the best comparison result for the corresponding metric.

4.5. Comparative Experiment

To validate the performance of the proposed model, RGC-YOLO was compared with several state-of-the-art object detection models, including YOLOv5s, Faster RCNN, SSD, and the lightweight YOLOv8-Ghost model, which is built using GhostNet as its backbone. The comparison results are summarized in Table 6.

Table 6. Training Results of Compared Models.

Model	Precision (%)	Recall (%)	mAP50 (%)	Parameters	GFLOPS ($\times 10^9$)	Size/MB	Inference Time (/ms)
RGC-YOLO	**88** *	**90.8** *	**93.2** *	2,011,659	5.8	4.21	6.6
YOLOv5	80.8	80.8	86.6	2,509,049	7.2	5.05	**6.3** *
Faster-RCNN	61	80.9	77.7	28,536,850	59.488	108	996.51
SSD	**88** *	81.1	88.4	26,285,486	62.798	91.7	149.64
YOLOv8-Ghost	86.5	78.6	88.4	**1,714,661** *	**5** *	**3.61** *	7.8

* Note: Bold values represent the best comparison result for the corresponding metric.

In terms of mAP50, RGC-YOLO achieved the highest performance, surpassing YOLOv5s, Faster RCNN, SSD, and YOLOv8-Ghost by 6.6, 15.5, 4.8, and 4.8 percentage points, respectively. Notably, for recall, RGC-YOLO achieved an impressive 90.8%, outperforming other models and effectively addressing the issue of missed detections.

Regarding precision, RGC-YOLO and SSD both achieved a value of 88%, the highest among all models. In comparison, YOLOv5s, Faster RCNN, and YOLOv8-Ghost were 7.2, 27, and 1.5 percentage points lower than RGC-YOLO, respectively. These results highlight the superior overall performance of RGC-YOLO in terms of both precision and recall.

In terms of model parameters and floating-point operations per second (FLOPs), RGC-YOLO demonstrated a significant advantage over larger models such as Faster RCNN and SSD, with its parameter count being only 1/14 and 1/13 of these models, respectively. Similarly, the FLOPs of RGC-YOLO were approximately one-tenth of those of Faster RCNN and SSD. For memory usage, RGC-YOLOs weight file size was only 4.21 MB, considerably smaller than Faster RCNN (108 MB) and SSD (91.7 MB). Although YOLOv8-Ghost used 14.25% less memory, 13.79% fewer FLOPs, and 14.76% fewer parameters than RGC-YOLO, its accuracy was notably lower, especially in recall, which reached only 78.6%. Such a low recall rate can lead to severe missed detection issues in real-time scenarios.

In terms of inference time, the Faster-RCNN model takes the longest time for a single image, approximately 151 times longer than the RGC-YOLO model. The SSD model has considerably reduced inference time compared to Faster-RCNN, but it still takes approximately 23 times longer. Among the YOLO series models, YOLOv5s inference time per image is 0.3 milliseconds shorter than RGC-YOLO, but in terms of accuracy, YOLOv5 performs worse on all metrics, with a recall rate 10 percentage points lower than RGC-YOLO. Although the YOLOv8-Ghost model has fewer parameters and GFLOPS, its inference time per image is longer than RGC-YOLO. This is due to the BN layer in the repghost module of RGC-YOLO, which merges parameters into the convolutional layers during inference, thus reducing memory usage.

We compared RGC-YOLO with existing models such as Faster R-CNN and SSD and found that RGC-YOLO not only has a significant advantage in accuracy but also features smaller model parameters and computational cost, along with a shorter inference time. In terms of inference time per image, the Faster-RCNN and SSD models are approximately 151 times and 23 times slower than RGC-YOLO, respectively. Furthermore, compared to other models in the YOLO series, such as YOLOv5 and YOLO-GHOST, RGC-YOLO exhibits superior recognition accuracy. Specifically, it outperforms YOLOv5 and YOLOv8-Ghost by 10 and 12.2 percentage points in recall rate, respectively. Moreover, the differences in parameter size and computational cost are minimal, enabling RGC-YOLO to maintain a lightweight design while notably improving accuracy.

4.6. Deployment Experiment

The above experimental results show that the lightweight improvements of RGC-YOLO are significant, reducing the computational cost and the number of parameters of the original model, thereby alleviating the computational burden on the hardware. To evaluate the performance of the lightweight improvements on a low-performance

computing device, deployment tests were conducted using the NVIDIA Jetson Nano as the deployment platform. The original model YOLOv8n and the improved model RGC-YOLO were selected, and the best weight files obtained from training were used to perform inference tests on the test set. The results of the experiments are shown in Table 7. On a desktop computer, the improved model reduced the prediction time for a single image by approximately 33.74%, while on the embedded platform, the improved model reduced the inference time for a single image by about 21.3%. Compared to the original model, the improved model has a shorter computation time on different computing devices, making it more suitable for devices with low computational resources and better adapted to real-world field scenarios.

Table 7. Comparison of inference time before and after model improvement.

Calculating Device	Model	Inference Time (/ms)
Computer terminal	YOLOv8n	10.6
	RGC-YOLO	**6.6** *
Jetson Nano	YOLOv8n	216
	RGC-YOLO	**170** *

* Note: Bold values represent the best comparison result for the corresponding metric.

A 1080P 2-megapixel high-definition camera was selected as the image acquisition device, combined with the NVIDIA Jetson Nano to build a real-time detection system. The real-time detection results are shown in the upper right corner of Figure 10. The real-time detection speed (150–170 ms/image) is consistent with the prediction speed on the dataset (170 ms/image).

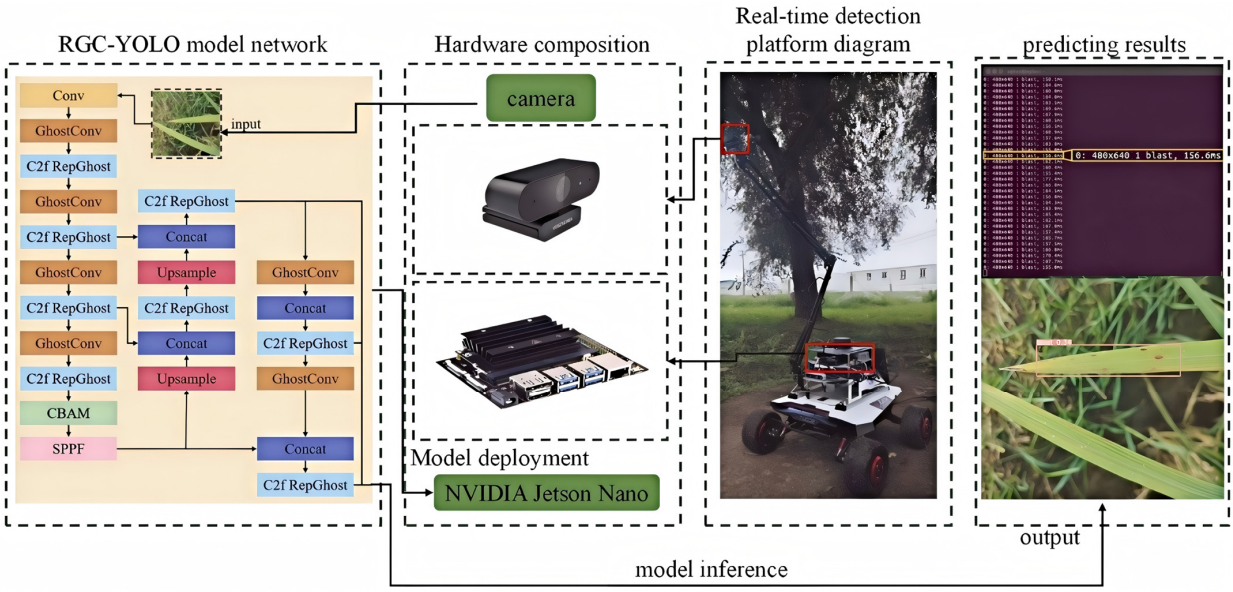


Figure 10. Real-time detection system and schematic diagram of detection results. Note: The red rectangular area represents the position of the hardware camera and NVIDIA Jetson Nano in the overall schematic diagram; The yellow rectangular box represents the real-time detection information output by the real-time monitoring system, which includes the following content: (camera number, image size, detected disease type, real-time detection time for a single image).

5. Conclusions

This research presents the innovative RGC-YOLO model, which achieves multi-scale rice pest and disease recognition with embedded device deployment for field use and high-accuracy real-time monitoring. The model is based on the YOLOv8n network with an SPPF layer and introduces GhostConv to replace traditional convolutional layers, uses the

reparameterized RepGhost module to replace the C2f layers, and incorporates the hybrid attention mechanism. These improvements effectively compensate for the accuracy loss caused by skipping connections and simple feature map duplication, thereby enhancing the recall rate. The main conclusions are summarized as follows:

1. Model Accuracy Evaluation: The RGC-YOLO model achieves a mAP50 of 0.932, a recall rate of 0.908, and an accuracy of 0.88. Compared to the base YOLOv8n model before the improvements, RGC-YOLO shows a significant improvement in missed detection performance.
2. Model Size Optimization: The RGC-YOLO model has 2.01 million parameters, 5.8×10^9 FLOPs, and a model size of 4.4 MB, representing reductions of 33.2%, 29.27%, and 26.42%, respectively, compared to the base YOLOv8n model.
3. Embedded Platform Deployment: The embedded platform deployment experiment further verifies the lightweight performance of the RGC-YOLO model. On the embedded device, the RGC-YOLO model achieves an image processing rate of 170 ms per image, which is a 40 ms improvement over the original YOLOv8n algorithm. This demonstrates that the RGC-YOLO model not only has a shorter inference time but also maintains high recognition accuracy and good recognition performance, making it suitable for real-time detection tasks.

Compared to other mainstream models such as Faster R-CNN, SSD, and YOLO series models, RGC-YOLO effectively balances detection accuracy and speed. The RGC-YOLO model has advantages for deployment and operation on low-resource devices. This research provides a feasible technical framework for the future development of agricultural monitoring technologies. Future work can further optimize the model, expand its application in various agricultural scenarios, and explore broader cross-domain applications.

Author Contributions: Conceptualization, J.W., S.M. and Y.W.; Data curation, J.W., S.M., Z.W. and G.C.; Formal analysis, J.W., S.M. and X.M.; Funding acquisition, J.W.; Investigation, J.W., Z.W., X.M., C.Y., G.C. and Y.W.; Methodology, J.W. and Y.W.; Project administration, S.M.; Resources, J.W.; Software, J.W., C.Y. and G.C.; Supervision, J.W. and Y.W.; Test, J.W., Z.W. and X.M.; Validation, Y.W.; Visualization, X.M., C.Y. and Y.W.; Writing—original draft, J.W.; Writing-review and editing, S.M.; All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Heilongjiang Provincial Natural Science Foundation of China, grant number (ZL2024E001); and the National Key R&D Program of China, grant number (2024YFD2001103).

Data Availability Statement: Data will be made available on request.

Acknowledgments: The authors express their gratitude to the Heilongjiang Provincial Key Laboratory of Modern Agricultural Equipment Technology in Northern Cold Regions for its support. Special thanks are extended to the villagers of Hongxing Village in Fangzheng County and the students from the College of Agriculture for their assistance during data collection. The authors also sincerely thank their laboratory colleagues for their valuable help.

Conflicts of Interest: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Lu, Y.; Tang, Y.; Zhang, J.; Liu, S.; Liang, X.; Li, M.; Li, R. Variations and Trends in Rice Quality across Different Types of Approved Varieties in China, 1978–2022. *Agronomy* **2024**, *14*, 1234. [CrossRef]
2. Wang, Z.; Chen, G.; Jiang, R.; Zhao, M.; Lin, T.; Wang, R.; Wang, J. SC-HybridSN: A Deep Learning Network Method for Rapid Discriminant Analysis of Industrial Paraffin Contamination Levels in Rice. *J. Food Compos. Anal.* **2024**, *133*, 106404. [CrossRef]

3. Zheng, Q.; Huang, W.; Xia, Q.; Dong, Y.; Ye, H.; Jiang, H.; Chen, S.; Huang, S. Remote Sensing Monitoring of Rice Diseases and Pests from Different Data Sources: A Review. *Agronomy* **2023**, *13*, 1851. [CrossRef]
4. Liao, J.; Tao, W.Y.; Zang, Y.; Zeng, H.Y.; Wang, P.; Luo, X. Research Progress and Prospect of Key Technologies in Crop Disease and Insect Pest Monitoring. *Trans. Chin. Soc. Agric. Mach.* **2023**, *11*, 1–19.
5. Wang, J.; Lin, T.; Ma, S.; Ju, J.; Wang, R.; Chen, G.; Jiang, R.; Wang, Z. The Qualitative and Quantitative Analysis of Industrial Paraffin Contamination Levels in Rice Using Spectral Pretreatment Combined with Machine Learning Models. *J. Food Compos. Anal.* **2023**, *121*, 105430. [CrossRef]
6. Pantazi, X.E.; Moshou, D.; Tamouridou, A.A. Automated Leaf Disease Detection in Different Crop Species through Image Features Analysis and One Class Classifiers. *Comput. Electron. Agric.* **2019**, *156*, 96–104. [CrossRef]
7. Shin, J.; Chang, Y.K.; Heung, B.; Nguyen-Quang, T.; Price, G.W.; Al-Mallahi, A. Effect of Directional Augmentation Using Supervised Machine Learning Technologies: A Case Study of Strawberry Powdery Mildew Detection. *Biosyst. Eng.* **2020**, *194*, 49–60. [CrossRef]
8. Abbaspour-Gilandeh, Y.; Aghabara, A.; Davari, M.; Maja, J.M. Feasibility of Using Computer Vision and Artificial Intelligence Techniques in Detection of Some Apple Pests and Diseases. *Appl. Sci.* **2022**, *12*, 906. [CrossRef]
9. Li, D.; Wang, R.; Xie, C.; Liu, L.; Zhang, J.; Li, R.; Wang, F.; Zhou, M.; Liu, W. A Recognition Method for Rice Plant Diseases and Pests Video Detection Based on Deep Convolutional Neural Network. *Sensors* **2020**, *20*, 578. [CrossRef]
10. Huang, S.P.; Sun, C.; Qi, L.; Ma, X.; Wang, W.J. Rice Panicle Blast Identification Method Based on Deep Convolution Neural Network. *Trans. Chin. Soc. Agric. Eng.* **2017**, *20*, 169–176.
11. Lin, X.Z.; Zhu, S.H.; Zhang, J.Y.; Liu, D.Y. Rice Planthopper Image Classification Method Based on Transfer Learning and Mask R-CNN. *Trans. Chin. Soc. Agric. Mach.* **2019**, *7*, 201–207.
12. Bari, B.S.; Islam, M.N.; Rashid, M.; Hasan, M.J.; Razman, M.A.M.; Musa, R.M.; Ab Nasir, A.F.; Majeed, A.P.A. A Real-Time Approach of Diagnosing Rice Leaf Disease Using Deep Learning-Based Faster R-CNN Framework. *PeerJ Comput. Sci.* **2021**, *7*, e432. [CrossRef] [PubMed]
13. Zhang, Y.; Zhong, L.; Ding, Y.; Yu, H.; Zhai, Z. ResViT-Rice: A Deep Learning Model Combining Residual Module and Transformer Encoder for Accurate Detection of Rice Diseases. *Agriculture* **2023**, *13*, 1264. [CrossRef]
14. Haruna, Y.; Qin, S.; Mbyamm Kiki, M.J. An Improved Approach to Detection of Rice Leaf Disease with GAN-Based Data Augmentation Pipeline. *Appl. Sci.* **2023**, *13*, 1346. [CrossRef]
15. Ju, J.; Chen, G.; Lv, Z.; Zhao, M.; Sun, L.; Wang, Z.; Wang, J. Design and Experiment of an Adaptive Cruise Weeding Robot for Paddy Fields Based on Improved YOLOv5. *Comput. Electron. Agric.* **2024**, *219*, 108824. [CrossRef]
16. Jiao, Z.; Huang, K.; Jia, G.; Lei, H.; Cai, Y.; Zhong, Z. An Effective Litchi Detection Method Based on Edge Devices in a Complex Scene. *Biosyst. Eng.* **2022**, *222*, 15–28. [CrossRef]
17. Lawal, O.M. YOLOv5-LiNet: A Lightweight Network for Fruits Instance Segmentation. *PLoS ONE* **2023**, *18*, e0282297. [CrossRef]
18. Firdiantika, I.M.; Lee, S.; Bhattacharyya, C.; Jang, Y.; Kim, S. EGCY-Net: An ELAN and GhostConv-Based YOLO Network for Stacked Packages in Logistic Systems. *Appl. Sci.* **2024**, *14*, 2763. [CrossRef]
19. Wu, C.; Ye, M.; Zhang, J.; Ma, Y. YOLO-LWNet: A Lightweight Road Damage Object Detection Network for Mobile Terminal Devices. *Sensors* **2023**, *23*, 3268. [CrossRef]
20. Huang, D.; Tu, Y.; Zhang, Z.; Ye, Z. A Lightweight Vehicle Detection Method Fusing GSConv and Coordinate Attention Mechanism. *Sensors* **2024**, *24*, 2394. [CrossRef]
21. Bouke, M.A.; Zaid, S.A.; Abdullah, A. Implications of Data Leakage in Machine Learning Preprocessing: A Multi-Domain Investigation. *Res. Sq.* **2024**. [CrossRef]
22. Haikal, A.L.A.; Yudistira, N.; Ridok, A. Comprehensive Mixed-Based Data Augmentation for Detection of Rice Leaf Disease in the Wild. *Crop Prot.* **2024**, *184*, 106816. [CrossRef]
23. Fan, Y.; Zhang, L.; Li, P. A Lightweight Model of Underwater Object Detection Based on YOLOv8n for an Edge Computing Platform. *J. Mar. Sci. Eng.* **2024**, *12*, 697. [CrossRef]
24. Cao, J.; Bao, W.; Shang, H.; Yuan, M.; Cheng, Q. GCL-YOLO: A GhostConv-Based Lightweight YOLO Network for UAV Small Object Detection. *Remote Sens.* **2023**, *15*, 4932. [CrossRef]
25. Chen, C.; Guo, Z.; Zeng, H.; Xiong, P.; Dong, J. RepGhost: A Hardware-Efficient Ghost Module via Re-Parameterization. *arXiv* **2024**. [CrossRef]
26. Ullah Khan, R.; Sheng Wong, W.; Ullah, I.; Algarni, F.; Inam Ul Haq, M.; Hardyman Bin Barawi, M.; Asghar Khan, M. Evaluating the Efficiency of CBAM-Resnet Using Malaysian Sign Language. *Comput. Mater. Contin.* **2022**, *71*, 2755–2772. [CrossRef]

27. Wang, J.; Chu, Y.; Chen, G.; Zhao, M.; Wu, J.; Qu, R.; Wang, Z. Characterization and Identification of NPK Stress in Rice Using Terrestrial Hyperspectral Images. *Plant Phenomics* **2024**, *6*, 0197. [CrossRef]
28. Ben Sada, A.; Khelloufi, A.; Naouri, A.; Ning, H.; Dhelim, S. Energy-Aware Selective Inference Task Offloading for Real-Time Edge Computing Applications. *IEEE Access* **2024**, *12*, 72924–72937. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.